tidyfinance: Transparent Factor Construction for Empirical Asset Pricing

by Christoph Frey, Christoph Scheuch, Stefan Voigt, and Patrick Weiss

Abstract The R package tidyfinance provides research infrastructure for the empirical asset pricing data workflow, which turns raw accounting and return data into factor portfolios for asset pricing tests, portfolio allocation, and the design of trading strategies. The package exposes every construction decision as a configurable parameter with a documented default. A unified download_data() interface covers Wharton Research Data Services (CRSP and Compustat), Ken French's Data Library, the Federal Reserve Economic Data, and other providers, applying provider-specific cleaning. Modular functions automate fiscal-year lagging, breakpoint computation, and portfolio aggregation, so that a default call reproduces standard practice while overriding any default takes a single argument. We illustrate the package by reproducing the asset growth factor of Cooper, Gulen, and Schill (2008) end to end from raw data, and by varying construction choices to show how they move the resulting premium.

1 Introduction

Factor returns enter empirical asset pricing tests, optimal portfolio allocation, and risk-management decisions. Constructing them typically involves a three-stage pipeline: data acquisition, usually downloading and cleaning raw return and accounting data from the Center for Research in Security Prices (CRSP) and Compustat; variable construction, including appropriate lags to avoid look-ahead bias; and portfolio sorting, where securities are assigned to portfolios based on characteristics. Each stage involves consequential design choices such as how to handle delistings, which fiscal-year lag to apply, where to set breakpoints, and whether portfolios should be equal- or value-weighted. Few pipelines are standardized, and fewer still ship with executable code. Therefore, reproducing or extending a factor series often starts with reconstructing the pipeline from scratch, with no way to verify that the result matches the original approach.

No widely adopted tool covers this upstream workflow. The most popular factor sources publish *series* without the code that produced them: Ken French's Data Library (Fama and French, 2023) supplies the Fama-French three- and five-factor returns and dozens of sorted portfolios. Some other projects provide code: the Open Source Bond Asset Pricing project (Dickerson et al., 2023), covers fixed income; the Open Source Cross-Sectional Asset Pricing repository (Chen and Zimmermann, 2022) reproduces 213+ published anomalies, and Jensen et al. (2023) provide a global factor library spanning hundreds of long-short signals across many countries. None of these sources, however, exposes the preprocessing pipeline as configurable code that researchers can modify. Many individual authors also host their own published factors, with transparency about construction choices varying widely. Walter et al. (2024) and Soebhag et al. (2024) show that these choices significantly affect the outcome of portfolio sorts. Existing R packages for quantitative finance operate downstream of factor construction, i.e., on price and return panels that the user must already have. Between raw data and standard asset-pricing tests sits an opaque layer, where construction choices accumulate silently.

This paper introduces **tidyfinance**, an R package on CRAN that exposes each stage of the pipeline as configurable code. The package addresses each stage with a dedicated interface. A unified `download_data()` function cleans, aligns, and harmonizes panels from Wharton Research Data Services (WRDS), Ken French's Data Library, Federal Reserve Economic Data (FRED), and seven other providers, including current constituents for 20 major equity indexes; Table 2 catalogs the full set. The WRDS panels cover CRSP, Compustat, the Trade Reporting and Compliance Engine (TRACE), and the Mergent Fixed Income Securities Database (FISD). Composable helpers then automate fiscal-year lagging, breakpoint computation, and portfolio aggregation. Every construction choice is exposed as a function argument with a default that follows established convention (for instance, value-weights and the Fama-French fiscal-year lag). A default call reproduces standard practice; overriding any default takes one argument.

We illustrate the package by reproducing a version of the asset growth factor of Cooper et al. (2008) end-to-end from raw CRSP and Compustat data, then varying any construction choice with a single-argument change. The package operationalizes the workflow developed in the open-access textbook *Tidy Finance with R* (Scheuch et al., 2023), which presents each step in tutorial form with inline scripts. The contribution of this paper is **tidyfinance** itself: a unified, configurable, tested, and versioned R package for this workflow. Where the book teaches a reader how to assemble the pipeline from scratch, the package supplies it as a single dependency that researchers can call, audit, and reuse without rebuilding the underlying code.

2 Background and related work

The R ecosystem for empirical asset pricing splits cleanly into three groups. Each addresses one slice of the workflow and leaves the rest to the user.

Data-acquisition packages. Several R packages retrieve specific data sources commonly used in asset pricing. **frenchdata** (Areal, 2021) downloads and parses the zip archives from Ken French's Data Library into long data frames; it does not touch WRDS and does not implement any construction step beyond reading the published factor series. **quantmod** (Ryan and Ulrich, 2025) and **tidyquant** (Dancho and Vaughan, 2026) cover Yahoo Finance, FRED, and Tiingo, returning daily open, high, low, and close price series and a small set of derived return helpers; neither connects to WRDS, and neither covers factor-construction logic. **Rblpapi** (Armstrong et al., 2024) queries the Bloomberg terminal for subscribers; the returned series leave the same factor-construction gap unfilled as the packages above. None of these packages covers the WRDS-CRSP-Compustat panel or the cleaning, alignment, and survivorship conventions the asset-pricing literature requires.

Downstream analysis packages. Once a cleaned panel is in hand, several packages support analysis. **PerformanceAnalytics** (Peterson and Carl, 2026) computes performance and risk metrics for return series. **fPortfolio** (Wuertz et al., 2023) covers portfolio optimization. **RQuantLib** (Eddelbuettel et al., 2024) interfaces with QuantLib for derivative pricing. **highfrequency** (Boudt et al., 2024) cleans intraday trade-and-quote panels and implements microstructure-volatility estimators. Each of these tools assumes that the panel of returns and factors is already constructed; none implements a factor-construction layer.

Factor data releases. A group of projects publishes *factor return series* directly. Ken French's Data Library (Fama and French, 2023) supplies the canonical Fama and French (1993) three- and five-factor returns and dozens of sorted portfolios, but documents construction in prose only and provides no associated code. Akey et al. (2026) show that these series change retroactively as the underlying method is silently revised, so that the factor returns a researcher obtains depend on *when* they were downloaded, with material effects on mutual-fund performance and cross-sectional asset pricing tests. A transparent, code-first pipeline removes precisely this opacity. Three more recent projects pair the published factor data with the construction script that produced it. Hou et al. (2020) replicate 452 cross-sectional anomalies and document that most fail out-of-sample, attributing the failures in part to undocumented construction variation across studies. The Open Source Cross-Sectional Asset Pricing repository (Chen and Zimmermann, 2022) responds by publishing per-anomaly construction scripts alongside the resulting 213+ long-short returns. The Open Source Bond Asset Pricing project (Dickerson et al., 2023) does the same for corporate bond factors. Jensen et al. (2023) provide the Global Factor Data of hundreds of long-short signals across many countries, together with the construction code in Python. These projects represent a substantial move toward transparency: a reader can inspect exactly how each factor was built and reproduce its returns. Two limitations remain. First, none exposes the pipeline as a callable API. A user who wants the asset growth factor rebalanced in January instead of July, or formed on NYSE-plus-NASDAQ breakpoints, or computed on a different fiscal-year lag, must fork the published script and edit it by hand. The script is not a function with named arguments. Second, the factor returns are frozen at the release date, with no in-workflow path to refresh them as new observations arrive or as a researcher's design choices evolve.

tidyfinance fills the missing gap: the package supplies the cleaned WRDS-CRSP-Compustat panel that data-acquisition packages do not produce and exposes the factor-construction pipeline that downstream analysis packages assume to be in hand. One `download_data()` call returns a cleaned WRDS-CRSP-Compustat panel; further functions implement the full factor-construction pipeline (fiscal-year lagging, breakpoint computation, portfolio sorting, long-short aggregation) as a chain of composable steps whose arguments cover the construction choices that vary across studies in the literature above. Two design principles, both inherited from Scheuch et al. (2023), guide the API. First, each construction decision is *transparent*: it appears as a configurable parameter with a documented default, in line with the tidy tools principles of Wickham (2023). Second, the pipeline is *modular* by construction: each construction step is implemented as a narrowly scoped function with clearly defined inputs and outputs. Researchers can therefore either chain these functions into a complete factor-construction workflow or reuse individual components within their own pipelines. **tidyfinance** additionally covers asset pricing tests and optimal portfolio choice; this paper focuses on factor construction and defers the remaining functionality to dedicated treatments.

3 Package overview

Installation and setup

You can install the released version of **tidyfinance** from CRAN via:

```
install.packages("tidyfinance")
```

To install the development version of **tidyfinance** from GitHub, first install **pak**, then run:

```
install.packages("pak")
pak::pak("tidy-finance/r-tidyfinance")
```

Once installed, load the package with:

```
library("tidyfinance")
```

Design and workflow

The empirical asset pricing workflow decomposes into three stages: data acquisition, variable construction, and portfolio sorting. Each stage involves dozens of small decisions: sample filters, cleaning rules, rebalancing conventions, and breakpoint definitions. Table 1 catalogs the user-facing functions exported by **tidyfinance**, grouped by workflow stage. The remainder of this section walks through them in narrative order.

Table 1: User-facing functions in **tidyfinance**, grouped by workflow stage. Functions whose first argument is a panel chain naturally via the native pipe `|>`; `filter_options()`, `breakpoint_options()`, `portfolio_sort_options()`, and `data_options()` (explained in detail below) bundle the configurable arguments for the sorting helpers.

Function	Purpose
Data acquisition	
`download_data()`	Retrieves and cleans a panel from supported data providers.
`set_wrds_credentials()`	Stores WRDS username and password in `.Renviron` for subsequent calls.
Variable construction	
`add_lagged_columns()`	Attaches the most recent observed value within a configurable look-back window, with optional Fama-French timing.
`join_lagged_values()`	Merges lagged values from a source panel into a target panel using the same windowing logic.
`winsorize()`	Replaces values outside the q-th and $(1-q)$-th percentiles with the corresponding quantile.
`trim()`	Sets values outside the q-th and $(1-q)$-th percentiles to `NA`.
`compute_rolling_value()`	Applies an aggregation function over a calendar-time rolling window.
`filter_sorting_data()`	Applies the sample filters bundled in `filter_options()`.
Portfolio sorting	
`compute_breakpoints()`	Computes quantile cutoffs from a cross-section, optionally restricted by exchange or by minimum market-cap quantile.
`assign_portfolio()`	Maps each observation to a portfolio integer using supplied or computed breakpoints.
`compute_portfolio_returns()`	Forms portfolios period by period and returns equal-, value-, and capped-value-weighted return panels.
`compute_long_short_returns()`	Computes the top-minus-bottom (or bottom-minus-top) spread from the output of `compute_portfolio_returns()`.
`implement_portfolio_sort()`	Wraps filtering, breakpoint computation, assignment, and aggregation into a single call for univariate and bivariate sorts.
Asset pricing tests	

Function	Purpose
`estimate_fama_macbeth()`	Runs cross-sectional ordinary least squares (OLS) regressions each period and aggregates into average risk premia with Newey and West (1987) standard errors.
`estimate_betas()`	Estimates rolling-window CAPM betas for each asset.
`estimate_model()`	Fits a linear model from a formula string and returns its coefficients as a single-row data frame, one column per coefficient.

Data acquisition

tidyfinance exposes one interface, `download_data()`, to 329 datasets: equity returns, fixed-income transactions, macroeconomic predictors, index constituents, and more. Table 2 lists all available sources. `list_supported_datasets()` returns the full catalog programmatically.

Table 2: Data sources available via `download_data()` in **tidyfinance**. For every dataset with a time dimension, the same `start_date` and `end_date` arguments subset the returned panel to a user-specified window.

Source	Description	Example
Public data sources		

Ken French's Data Library	Fama-French 169+ factor models, portfolios, industry classifications	`download_data(`
		`  domain = "Fama-French",`
		`  dataset = "Fama/French 3 Factors")`

Global Q	Hou et al. (2021) q5 factor model (market, size, investment, ROE, expected growth) at multiple frequencies	`download_data(`
		`  domain = "Global Q",`
		`  dataset = "factors_q5_monthly"`
		`)`

Stambaugh-Yuan	Stambaugh and Yuan (2017) mispricing factors (MGMT, PERF) at daily and monthly frequency	`download_data(`
		`  domain = "Stambaugh-Yuan",`
		`  dataset = "monthly"`
		`)`

Pastor-Stambaugh	Pástor and Stambaugh (2003) traded and non-traded liquidity factors	`download_data(`
		`  domain = "Pastor-Stambaugh"`
		`)`

Global Factor Data	Jensen et al. (2023) hundreds of long-short factors across many countries and asset classes	`download_data(`
		`  domain = "Global Factor Data",`
		`  dataset = "factors"`
		`)`

Goyal-Welch Predictors	Macroeconomic return predictors	`download_data(`
		`  domain = "Goyal-Welch",`
		`  dataset = "monthly"`
		`)`

FRED	St. Louis Fed economic time series (GDP, CPI, etc.)	`download_data(`
		`  domain = "FRED",`
		`  series = "GDP"`
		`)`

Yahoo Finance	Historical price data with adjusted close for any ticker	`download_data(`
		`  domain = "Stock Prices",`
		`  symbols = "AAPL"`
		`)`

Index Constituents	Current members of major equity indexes (S&P 500, Russell, etc.)	`download_data(`
		`  domain = "Index Constituents",`
		`  index = "DAX"`
		`)`

Open Source Asset Pricing	213+ monthly asset pricing signals and anomalies	`download_data(`
		`  domain = "Open Source Asset Pricing"`
		`)`

High-Frequency	5-second frequency data for S&P 500 constituents constructed by Hautsch et al. (2026)	`download_data(`
		`  domain = "Tidy Finance",`
		`  dataset = "high_frequency_sp500"`
		`)`

Source	Description	Example

Risk-Free Rate	Daily and monthly risk-free rates (derived from FRED T-bill series by **tidyfinance**)	`download_data(`
		`  domain = "Tidy Finance",`
		`  dataset = "risk_free"`
		`)`

Factor Library	Precomputed factor returns hosted on Hugging Face Hub	`download_data(`
		`  domain = "Tidy Finance",`
		`  dataset = "factor_library",`
		`  sorting_variable = "ag"`
		`)`

WRDS (subscription required)		

CRSP	Daily and monthly stock returns, market cap, listing exchange, industry information	`download_data(`
		`  domain = "WRDS",`
		`  dataset = "crsp_monthly"`
		`)`

Compustat	Annual and quarterly financial statements	`download_data(`
		`  domain = "WRDS",`
		`  dataset = "compustat_annual"`
		`)`

CCM Links	CRSP-Compustat linking table (permno to gvkey)	`download_data(`
		`  domain = "WRDS",`
		`  dataset = "ccm_links"`
		`)`

FISD	Mergent Fixed Income Securities Database	`download_data(`
		`  domain = "WRDS",`
		`  dataset = "fisd"`
		`)`

TRACE	Corporate bond transactions Dick-Nielsen (2009) cleaned	`download_data(`
		`  domain = "WRDS",`
		`  dataset = "trace_enhanced",`
		`  cusips = "655844CJ5"`
		`)`

A first illustration uses only public data sources. We retrieve the current constituents of the Dow Jones Industrial Average and then download daily prices from Yahoo Finance for those tickers over a chosen window. `list_supported_indexes()` returns the full set of equity indexes available through the `"Index Constituents"` domain.

```
dow_constituents <- download_data(
  domain = "Index Constituents",
  index = "Dow Jones Industrial Average"
)
download_data(
  domain = "Stock Prices",
  symbols = dow_constituents$symbol,
  start_date = "2020-01-01",
  end_date = "2024-12-31"
)
```

```
#> # A tibble: 37,710 x 8
#>    symbol date        volume  open   low  high close adjusted_close
#>    <chr>  <date>       <dbl> <dbl> <dbl> <dbl> <dbl>          <dbl>
#>  1 GS     2020-01-02 3736300  231   230.  235.  234.           201.
#>  2 GS     2020-01-03 2274500  232.  230.  233.  232.           199.
#>  3 GS     2020-01-06 3329300  230.  229.  234.  234.           201.
#>  4 GS     2020-01-07 5255200  235   235.  238.  235.           202.
#>  5 GS     2020-01-08 3564700  236.  235.  240.  238.           204.
#>  6 GS     2020-01-09 3980700  241.  240.  243.  243.           208.
#>  7 GS     2020-01-10 2248100  243.  241.  243.  242.           208.
#>  8 GS     2020-01-13 3359200  244.  243   246.  245.           211.
#>  9 GS     2020-01-14 4302800  245.  245.  249.  246.           211.
#> 10 GS     2020-01-15 5411200  242   239.  250.  245.           211.
#> # i 37,700 more rows
```

Similarly, we can fetch the Fama-French three-factor data from Ken French's Data Library with a single call to download and parse the published archives:

```
ff_factors <- download_data(
  domain = "Fama-French",
  dataset = "Fama/French 3 Factors",
  start_date = "1972-01-01",
  end_date = "2024-12-31"
)
```

The `start_date` and `end_date` arguments generalize across every dataset type in Table 2 that supports a time dimension.

Public data still requires nontrivial cleaning. The Goyal and Welch (2008) macroeconomic predictors are a typical case: `download_data(domain = "Goyal-Welch", dataset = "monthly")` fetches the raw spreadsheet at the requested frequency (set `dataset` to `monthly`, `quarterly`, or `annual`), parses dates, strips comma thousand separators, and coerces character columns to numeric. It then constructs the standard predictors used in the equity-premium prediction literature: the log dividend-price ratio (`dp`), dividend yield (`dy`), earnings-price ratio (`ep`), dividend-payout ratio (`de`), the forward equity risk premium (`rp_div`), the term spread (`tms`), and the default-yield spread (`dfy`). The output is the cleaned panel filtered to the requested date range.

For empirical asset pricing work, WRDS is the common source of return and accounting panels. WRDS data are proprietary and require an institutional subscription and credentials. **tidyfinance** handles this setup once. `set_wrds_credentials()` prompts for the username and password and stores them in a project-specific or global `.Renviron` file, and prompts the user to add `.Renviron` to `.gitignore` to prevent credential leakage. Subsequent `download_data()` calls open the database connection automatically. Depending on the user's WRDS account settings, `download_data()` may also trigger a two-factor authentication (2FA) challenge that WRDS sends to the registered device; once approved, the connection proceeds normally.

`download_data()` applies standard cleaning conventions following Bali et al. (2016) and Scheuch et al. (2023) automatically. For CRSP monthly data, the function restricts the universe to common equity securities listed on NYSE, AMEX (now NYSE American), or NASDAQ. It computes market capitalization as shares outstanding times price, constructs a one-month market capitalization lag for portfolio weights, and maps listing exchange codes and Standard Industrial Classification (SIC) codes to readable labels. Excess returns are raw returns minus a risk-free rate that **tidyfinance** constructs from FRED by splicing the 3-month Treasury bill secondary-market rate (pre-2001) with the 4-week Treasury bill secondary-market rate (from 2001 onward); the spliced series is hosted on Hugging Face and refreshed monthly by a scheduled GitHub Actions workflow. The function drops observations with missing returns or market capitalization, and incorporates delisting returns directly, eliminating the need for a separate adjustment. The function defaults to the CRSP 2.0 (CIZ) layout first introduced in July 2022. Alternatively, passing `version = "v1"` returns the legacy (SIZ) layout, which ends in December 2024. Schwarz et al. (2026) document that the move to CIZ rewrites a non-trivial share of historical monthly returns through a changed dividend reinvestment assumption, yet find that average premia and their significance are largely unaffected.

For Compustat annual data, `download_data()` restricts to industrial-format consolidated statements and retains only the most recent observation per firm-year. Following Fama and French (1992), it constructs book equity in priority order: stockholders' equity when available, else common equity plus preferred stock, else total assets minus total liabilities. The function adds back deferred taxes and investment tax credits, and subtracts the book value of preferred stock.

For TRACE corporate bond transactions, `download_data()` applies the Dick-Nielsen cleaning protocol (Dick-Nielsen, 2009) to remove cancellations, corrections, and reversals before returning the trade panel.

Across `download_data()`, every returned panel exposes a single `date` column of `<date>` type, regardless of source. Joining observations from different providers therefore reduces to an equality join on `date` without manual format conversion. The convention is the same across frequencies: daily observations return the trading date, monthly observations the first calendar day of the reference month (regardless of whether it was a trading day), and quarterly and annual observations the first day of the fiscal period-end month as reported by the original source. For Compustat, the latter is the firm's fiscal period-end (`datadate`). Macroeconomic series from FRED keep their original release dates. Where the underlying source uses an internal date that differs from this convention, **tidyfinance** preserves it as an auxiliary column: CRSP 2.0 monthly returns, for example, retain the trading-date `mthcaldt` as `calculation_date` alongside the harmonized `date`. Cross-frequency joins, such as merging monthly returns with annual fundamentals, require the lagging logic described in Section 2.3.4.

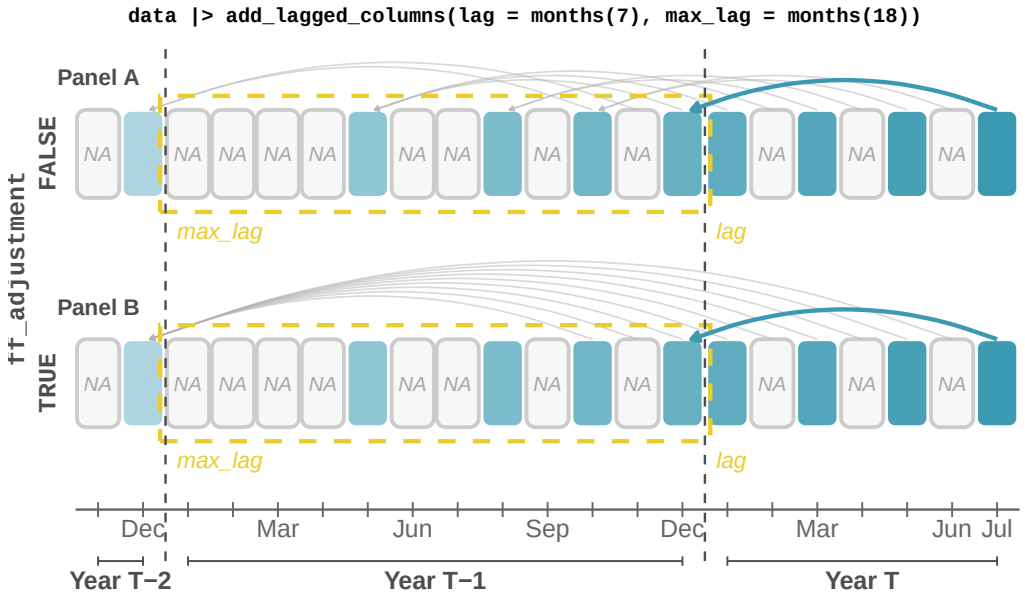


Figure 1: Two lagging mechanisms in add_lagged_columns(). Each row represents a data series with observed (filled) and missing (hollow) cells. The dashed rectangle marks the search window from the perspective of the most recent observation (the top row); arrows show which value gets matched to each target month. Panel A (ff_adjustment = FALSE) returns the most recent observed value within a window bounded by lag and max_lag. Panel B (ff_adjustment = TRUE) implements the Fama-French convention, where the lagged value updates only at the June/July boundary and is held constant for the next twelve months.

Variable construction

Most predictive variables in asset pricing studies use lagged information so that, for example, portfolio formation in month t uses only data observable at the end of month $t-1$. `add_lagged_columns()` automates the required lagging procedure. For each row in the data, it looks back over a configurable window and attaches the most recent observed value of the chosen columns. Figure 1 illustrates the two modes the function supports. By default (`ff_adjustment = FALSE`), shown in Panel A, the window is bounded by `lag` (the minimum delay between observation and use) and `max_lag` (the maximum delay before a stale value is treated as missing). Within this window (the dashed rectangle), the function picks the most recent non-missing value; if no observation falls in the window, the lagged column takes `NA`.

The second mode, enabled by `ff_adjustment = TRUE` and shown in Panel B of Figure 1, implements the Fama-French convention (Fama and French, 1993). The convention assumes accounting variables for fiscal year $t-1$ become public information by June of year t, then uses them for portfolio formation from July of t through June of $t+1$. This timing produces a step-function lag that updates once per year at the June/July boundary, regardless of the underlying fiscal year-end: all months from July through June share the value locked in at the previous rebalancing.

`join_lagged_values()` is a related helper that merges lagged values from one data frame into another using the same windowing logic. It is useful when the target panel and the source panel are at different frequencies, for example, when joining monthly accounting data into a quarterly target panel. The function takes the same `ff_adjustment` argument as `add_lagged_columns()`, so the Fama-French convention can be applied at the join step rather than as a separate transformation.

A handful of additional helpers cover routine operations that recur across empirical studies. `winsorize()` and `trim()` handle outliers in cross-sectional variables: the former symmetrically replaces values below the q-th and above the $(1-q)$-th percentile with the corresponding quantile values, while the latter sets them to `NA`. `estimate_model()` fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame, which composes naturally with `group_by()` for grouped or rolling regressions such as estimating time-series factor exposures stock by stock via `estimate_betas()`. `compute_rolling_value()` applies an arbitrary aggregation function over a rolling window defined in calendar time rather than row counts, accommodating irregular observation frequencies and missing periods. `filter_sorting_data()` screens a panel before sorting, dropping

observations that fail the sample filters bundled in a `filter_options()` object, such as financial or utility firms or stocks below a minimum price or size; by default it reports how many observations each filter removes, which `quiet = TRUE` suppresses.

Portfolio sorting

A portfolio sort groups stocks each period into ordered portfolios according to the level of a sorting variable. The boundaries between portfolios are called *breakpoints*: numeric cutoffs of the sorting variable that determine which portfolio a stock joins. A quintile sort on size, for instance, has four breakpoints at the 20th, 40th, 60th, and 80th percentiles of the cross-sectional market-capitalization distribution. Common sorting variables include size (measured as market capitalization), beta, book-to-market, and momentum. The long-short return between the extreme portfolios is interpreted as a factor-mimicking strategy and serves as the basis for tests of whether the sorting variable forecasts future returns. Portfolio sorts are among the most widely used tools in empirical asset pricing (Bali et al., 2016). We expose the procedure through four composable building blocks plus a one-call wrapper.

`compute_breakpoints()` takes a cross-section and a sorting variable and returns the numeric cutoffs that define the portfolio boundaries. The user supplies either a number of portfolios (for example, five for quintiles) or an explicit vector of percentiles, but not both. A `breakpoints_exchanges` argument restricts the universe used to compute breakpoints. The standard convention is NYSE-listed stocks only, which prevents the large cross-section of small NASDAQ-listed firms from dominating the breakpoints (Jensen et al., 2023). A related `breakpoints_min_size_threshold` argument excludes stocks below a market-cap quantile from the breakpoint computation, which size-related sorts often require because microcaps would otherwise distort the cutoffs. A `smooth_bunching` flag handles cases where the sorting variable takes the same value for many observations at the bottom or top of the distribution, so that the lowest (or highest) quantile breakpoints coincide. Without intervention, all tied observations would land in a single boundary portfolio while neighboring portfolios remain empty. When `smooth_bunching = TRUE`, the function assigns the bunched observations to the boundary portfolio and recomputes the remaining breakpoints on the subset of observations strictly inside the boundary value, restoring well-defined cutoffs for the interior portfolios.

`assign_portfolio()` maps each observation to a portfolio integer and uses breakpoints supplied either directly or via a breakpoint-computing function (defaulting to `compute_breakpoints()`). In practice, `assign_portfolio()` is called separately for each rebalancing date, so that portfolios are reformed each period rather than fixed once for the whole sample.

`compute_portfolio_returns()` ties the lower-level pieces together. Given a panel of stock data, one or two sorting variables, a sorting method, and `breakpoint_options`, it handles rebalancing, breakpoint computation, portfolio assignment, and return aggregation. It then returns a complete panel of portfolio returns. For each portfolio-date the function reports three return measures: equal-weighted, value-weighted using `mktcap_lag` as the weight, and capped value-weighted (Jensen et al., 2023). The capped version pulls weights above the `cap_weight` percentile (default `0.8` corresponds to 80%) back to that cap, which prevents a few megacap firms from dominating the portfolio. A `min_portfolio_size` argument sets the minimum number of constituents below which the corresponding return is set to `NA`. An optional `rebalancing_month` argument switches from period-by-period rebalancing to annual rebalancing in the specified month. Portfolios formed in that month are then held constant for the following twelve months.

The function supports three sorting schemes, illustrated in Figure 2. A *univariate* sort uses a single variable. An *independent bivariate* sort computes breakpoints on each variable separately and intersects the resulting bins to produce a two-way grid. A *dependent bivariate* sort first partitions the cross-section by the secondary (control) variable and then re-sorts within each bucket on the main variable, which is the appropriate choice when the second variable is treated as a control rather than as an independent dimension of variation. For bivariate sorts, returns are averaged over the secondary variable so that only the main-variable portfolios are returned.

`compute_long_short_returns()` takes the output of `compute_portfolio_returns()` and computes the spread between the top and bottom portfolios. A `direction` argument controls the sign and defaults to `"top_minus_bottom"`. The function returns one long-short series per return measure available in the input, so the equal-weighted, value-weighted, and capped value-weighted spreads are produced together. This long-short return is the factor-mimicking spread that the asset-pricing literature typically takes as the object of interest.

For users who want the full pipeline in a single call, `implement_portfolio_sort()` wraps the four functions above. It takes a panel of stock data, one or two sorting variables, a sorting method, and a configuration object, and returns the long-short factor return series alongside the underlying portfolio returns. The lower-level functions remain available for custom workflows that need finer control over

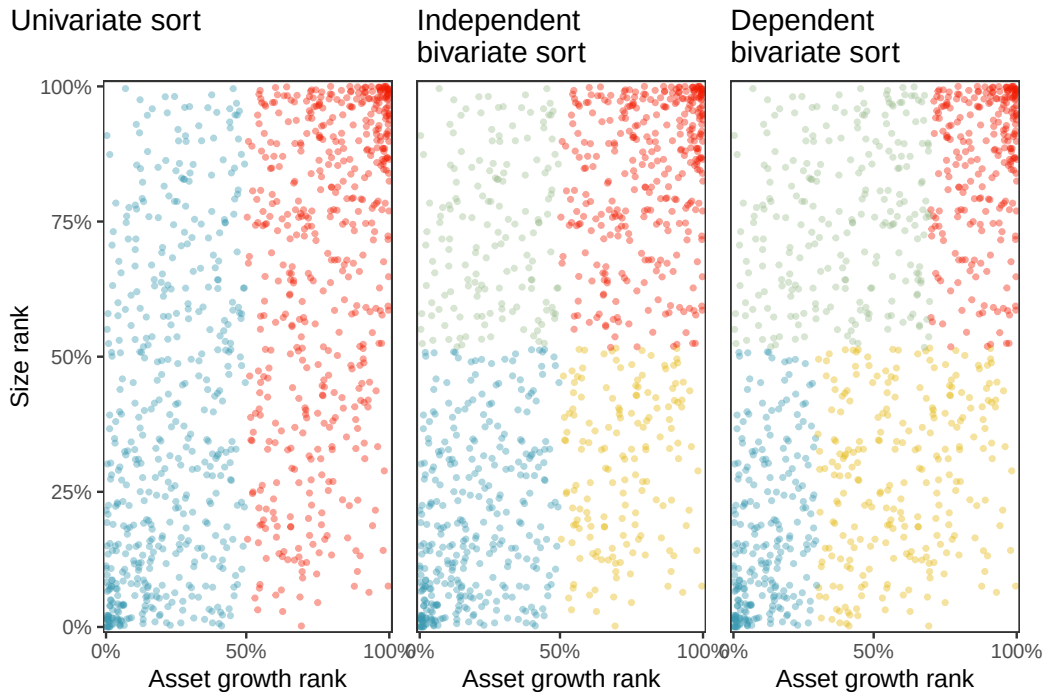


Figure 2: Portfolio sort approaches illustrated on a simulated cross-section. Each panel plots simulated observations assigned to 2 × 2 portfolios based on asset growth rank (x-axis) and size rank (y-axis), both expressed as percentiles. Colors indicate portfolio cell assignment. The univariate sort (left) assigns portfolios on asset growth alone; the independent bivariate sort (center) assigns each dimension independently; the dependent bivariate sort (right) sorts on size first, then on asset growth within size groups.

breakpoints, weighting, or rebalancing.

4 A case study in factor construction

We walk through the construction of the asset growth factor from raw CRSP and Compustat data to long-short portfolio returns, applying each stage of Table 1 in turn.

Load **tidyfinance** together with a minimal set of supporting packages: **dplyr** (Wickham et al., 2026) for data manipulation, **tidyr** (Wickham et al., 2024) for reshaping between long and wide formats, and **lubridate** (Spinu et al., 2024) for date arithmetic.

```
library("tidyfinance")
library("dplyr")
library("tidyr")
library("lubridate")
```

Data acquisition

Constructing the asset growth factor requires two WRDS panels: annual accounting data from Compustat to compute the asset growth signal and monthly stock returns from CRSP to translate the signal into portfolio returns.

CRSP and Compustat are proprietary and require a WRDS subscription. All numbers, tables, and figures in this paper are computed from the real WRDS data with `data_domain <- "WRDS"`.

```
data_domain <- "WRDS"
```

Readers without a WRDS subscription can render the entire manuscript against pseudo data of identical schema by setting `data_domain <- "Pseudo Data"` in the chunk below. The pseudo path covers the three WRDS-sourced datasets the case study consumes: monthly CRSP returns, annual

Compustat fundamentals, and the CRSP-Compustat linking table. Every `download_data()` call for one of these datasets then routes to an in-package pseudo generator rather than to WRDS, so the document compiles end-to-end without a subscription, credentials, or network access to a paid provider. The generator preserves column names, data types, missingness patterns, panel structure, and approximate scales, so the cleaning, lagging, and sorting code paths exercise the same branches they would on real data, and every figure and table in this paper renders without error. What the generator does not preserve are economic relations. Returns are independent draws around a calibrated mean, and characteristics carry no predictive content for subsequent returns; the long-short premia recovered against pseudo data are therefore noise.

Two arguments to `download_data()` control the pseudo draws: `seed` fixes the random number generator so that two readers running the same chunk obtain bit-identical pseudo panels, and `n_assets` sets the size of the pseudo cross-section (default `1000L`), trading off compile speed against the realism of cross-sectional statistics. The practical implication is that every chunk in this paper can be inspected, edited, and re-rendered on a laptop with no proprietary data and no credentials; only the headline factor premia require WRDS to reproduce numerically.

Before the first WRDS download, call `set_wrds_credentials()` to store WRDS credentials securely in `.Renviron`; subsequent `download_data()` calls establish the database connection automatically.

```
date_start <- as.Date("1972-01-01")
date_end <- as.Date("2024-12-31")
```

We restrict the sample to the period from 1972 onward because Compustat coverage is often incomplete before that date, and we end the sample at 2024, the most recent full calendar year at the time of writing.

First, we download monthly stock returns from CRSP. Setting `add_ccm_links = TRUE` appends CRSP-Compustat linking identifiers needed to merge with financial statement data in the next step.

```
crsp_monthly <- download_data(
  domain = data_domain,
  dataset = "crsp_monthly",
  version = "v2",
  start_date = date_start,
  end_date = date_end,
  add_ccm_links = TRUE
)
```

The dataset contains 3,176,966 stock-month observations for 25,347 unique stocks, covering the period from January 1st, 1972 to December 31st, 2024.

Next, we download annual financial statements from Compustat.

```
compustat_annual <- download_data(
  domain = data_domain,
  dataset = "compustat_annual",
  start_date = date_start,
  end_date = date_end,
  additional_columns = c("at", "ib"),
  only_usd = TRUE
) |>
  select(gvkey, date, at, ib)
```

Annual Compustat data contain 472,100 firm-year observations for 39,350 unique firms, covering the period from January 1st, 1972 to December 31st, 2024. The `additional_columns` argument retrieves total assets (`at`) and income before extraordinary items (`ib`), needed for computing asset growth and applying an earnings filter in later steps. Setting `only_usd = TRUE` restricts the dataset to USD-denominated stocks.

Variable construction

We illustrate data cleaning and variable construction using asset growth (AG) as our running example. Asset growth for a firm i in year t is the year-over-year percentage change in total assets (Cooper et al., 2008),

$$AG_{t,i} = (AT_{t,i} - AT_{t-1,i}) / AT_{t-1,i},$$

where $AT_{t,i}$ is total assets for firm i in year t (retrieved in column `at`).

The `add_lagged_columns()` function from **tidyfinance** handles temporal alignment by creating a lagged version of total assets grouped by firm identifier (`gvkey`) with a one-year lag. Non-finite values (arising when lagged total assets are zero or missing) become `NA`.

```
sorting_variable_data <- compustat_annual |>
  add_lagged_columns(
    cols = "at",
    lag = years(1),
    by = "gvkey"
  ) |>
  mutate(
    asset_growth = (at - at_lag) / at_lag,
    asset_growth = if_else(is.finite(asset_growth), asset_growth, NA_real_)
  ) |>
  select(gvkey, date, asset_growth, ib)
```

The call relies on two silent defaults: `max_lag` falls back to `lag` (here one year), so only the single observation exactly one year before each row is eligible; `ff_adjustment = FALSE`, so the look-back is the plain calendar window rather than the Fama-French June/July rebalancing convention introduced in Section 2.3.4.

`sorting_variable_data` contains one observation per firm-year, the natural unit for cross-sectional analysis of annual accounting variables. Asset growth is heavily right-skewed: a small number of firms with dramatic balance-sheet expansions generate extreme positive values. Table 3 reports the pooled cross-sectional summary alongside the time-series average, minimum, and maximum of the corresponding statistic computed cross-sectionally each year.[1] We obtain both views with `create_summary_statistics()`, a **tidyfinance** helper that returns the standard set of cross-sectional summary statistics for a variable: applied to the pooled (winsorized) sample for the first column and grouped by calendar year for the next three. Table 3 assembles the two computations side by side.

```
pooled_stats <- sorting_variable_data |>
  drop_na(asset_growth) |>
  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
  create_summary_statistics(asset_growth, detail = TRUE)

yearly_stats <- sorting_variable_data |>
  drop_na(asset_growth) |>
  group_by(year = year(date)) |>
  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
  ungroup() |>
  create_summary_statistics(asset_growth, by = year, detail = TRUE)
```

The next step merges annual sorting variables onto the monthly CRSP stock panel using `join_lagged_values()`. The function aligns fiscal-year-end data with monthly observations through a configurable lag window. Setting `ff_adjustment = TRUE` implements the Fama-French convention (Fama and French, 1993), aligning fiscal-year data from year t with portfolio formation in July of year $t + 1$ through a 12-to-18-month look-back window.

```
sorting_data <- crsp_monthly |>
  join_lagged_values(
    new_data = sorting_variable_data,
    id_keys = "gvkey",
    min_lag = months(7),
    max_lag = months(18),
    ff_adjustment = TRUE
  ) |>
  select(
    date,
```

[1] The pooled count of 358,039 firm-years is below the 472,100 downloaded in Section 2.4.1 because the one-year lag in the asset-growth construction drops each firm's first observation.

Table 3: Cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). The 'Pooled' column reports the statistic computed on the entire firm-year sample. The 'Yearly' columns summarize the time series of statistic values computed cross-sectionally year by year, taking the average, minimum, and maximum across calendar years. N is the count of firm-year observations (pooled) or per-year observations (yearly columns). Sample: 1973–2024.

	Statistic	Pooled	Yearly.avg	Yearly.min	Yearly.max
n	N	358,039	6,885	3,808	9,467
mean	Mean	0.249	0.261	0.048	0.889
sd	SD	0.932	0.980	0.203	3.830
q10	P10	-0.205	-0.196	-0.397	-0.007
q25	P25	-0.041	-0.039	-0.151	0.060
q50	Median	0.061	0.062	0.001	0.132
q75	P75	0.208	0.210	0.105	0.370
q90	P90	0.609	0.631	0.253	1.341

```
    permno,
    exchange,
    siccd,
    ret_excess,
    mktcap_lag,
    asset_growth,
    ib
  ) |>
  filter(date >= date_start + months(12) + months(18))
```

The filter removes the initial year used to compute the growth rate plus an additional 18 months to avoid mechanically introduced gaps from the lagging step, leaving a panel of stock-month observations with excess returns, exchange membership, lagged market capitalization, and the lagged sorting variable. The auxiliary columns `siccd` and `ib` support the sample-construction filters in Section 2.4.3.

Figure 3 shows the 33rd, 50th, and 66th cross-sectional percentiles of asset growth over time in the resulting panel; pronounced spikes during economic expansions and contractions highlight the cyclical nature of corporate investment.

With the sorting variable constructed and merged onto the monthly stock panel, we now turn to portfolio sorts, the mechanism that translates firm-level characteristics into tradable factor returns.

Portfolio sorting

In portfolio sorts, stocks are ranked each period by their sorting variable and divided into quantile portfolios. Portfolio returns are equal-weighted or value-weighted averages of constituent stock returns. The long-short factor return is the difference between the extreme portfolios. For asset growth, the relevant spread is bottom minus top, reflecting the negative premium documented by Cooper et al. (2008). The construction involves several methodological choices: which stocks enter the universe, how many portfolios to form, which exchange determines the breakpoints, the weighting scheme, and the rebalancing frequency.

`implement_portfolio_sort()` exposes all of these choices through a single function call. The function takes the panel data, one or two sorting variables, a sorting method, and a `portfolio_sort_options` object that bundles two configuration layers: `filter_options()` controls which observations enter the breakpoint and return computation (for example, excluding financial firms or stocks below a minimum price), and `breakpoint_options()` controls how portfolios are formed (number of portfolios, exchange universe, bunching). Internally, the function applies the sample filters and then calls `compute_portfolio_returns()`, which produces the time series of portfolio returns. A subsequent call to `compute_long_short_returns()` constructs the long-short portfolio.

`compute_portfolio_returns()` itself rests on two lower-level helpers, `compute_breakpoints()` for quantile cutoffs and `assign_portfolio()` for mapping each stock-month to a portfolio number. Both are exported, so researchers who need finer control can call them directly rather than through the wrapper. The two examples below use `implement_portfolio_sort()` throughout.

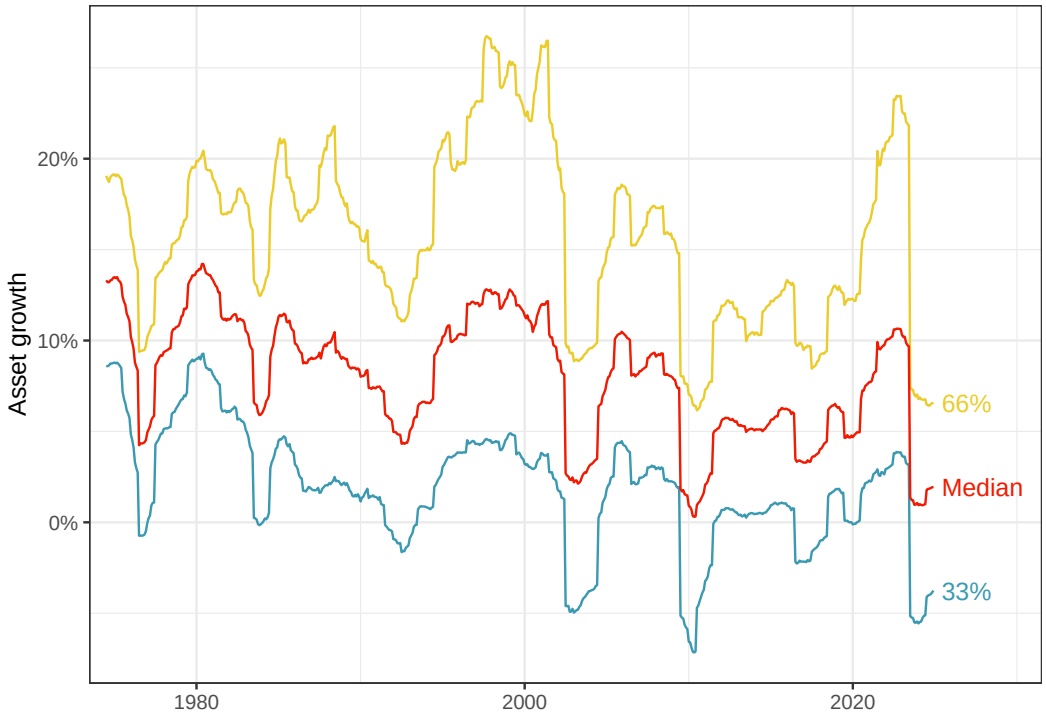


Figure 3: Selected percentiles (33rd, 50th, 66th) of cross-sectional asset growth over time. The y-axis reports annual growth in total assets in percent. Sample: 1974–2024.

Baseline specification (univariate sorts)

The simplest call to `implement_portfolio_sort()` requires only a `portfolio_sort_options` object that specifies how to form the breakpoints. In a univariate sort, stocks are ranked on a single characteristic. We illustrate using asset growth as the sorting variable, forming quintile portfolios with breakpoints computed from NYSE-listed stocks only, and rebalancing annually in July.

```
options_baseline <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  )
)
```

The `breakpoint_options()` call specifies that breakpoints come only from NYSE-listed stocks, a common convention that avoids the influence of small AMEX and NASDAQ firms on portfolio cutoffs. We then pass these options to `implement_portfolio_sort()`.

```
portfolio_returns <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = "asset_growth",
  sorting_method = "univariate",
  rebalancing_month = 7,
  portfolio_sort_options = options_baseline
)
```

Apart from setting `rebalancing_month = 7`, every other choice point falls back to the package default: a cap-weight cutoff at the 80th percentile of market capitalization for the capped value-weighted return, no minimum portfolio size, and no smooth bunching. The standard CRSP/Compustat column names (`permno`, `date`, `ret_excess`, `mktcap_lag`, `exchange`) are picked up automatically through the default `data_options()`. Setting `rebalancing_month = 7` reforms the portfolios each July and holds them for the following twelve months, in line with the standard Fama-French rebalancing convention. This schedule pairs with `ff_adjustment = TRUE` in `join_lagged_values()`: the signal is locked in at the June/July boundary, so July is the first month in which a fresh fiscal-year update is available.

Table 4: Time-series averages of annualized monthly excess returns for asset growth quintile portfolios. Portfolios are formed annually in July using NYSE breakpoints. Means are annualized (multiplied by 12). The *t* statistic tests whether the mean monthly excess return differs from zero. Capped value-weighted pulls weights above the 80th percentile back to that cap. The L/S row is the long-short portfolio (bottom minus top quintile). Sample: 1974–2024.

	Value-weighted		Equal-weighted		Capped value-weighted	
Portfolio	Mean	*t* statistic	Mean	*t* statistic	Mean	*t* statistic
1	0.110	4.582	0.162	4.682	0.117	4.105
2	0.094	4.585	0.132	5.228	0.111	4.692
3	0.085	4.027	0.125	5.069	0.109	4.711
4	0.086	3.792	0.116	4.444	0.103	4.152
5	0.076	2.686	0.060	1.776	0.068	2.182
L/S	0.034	2.280	0.102	8.345	0.048	4.137

`compute_portfolio_returns()`, called internally, returns a long data frame with one row per portfolio-month and the three weighting columns introduced in Section 2.3.5: `ret_excess_vw` (value-weighted, VW), `ret_excess_ew` (equal-weighted, EW), and `ret_excess_vw_capped` (capped value-weighted, VWC). Piping the result through `compute_long_short_returns()` gives the long-short portfolio. The `direction = "bottom_minus_top"` argument means the factor is long low-AG stocks and short high-AG stocks.

```
factor_returns_ag <- portfolio_returns |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 4 summarizes the time series of each quintile across all three weighting schemes, with the long-short portfolio appended as the final row.

```
return_summary <- function(data) {
  data |>
    drop_na() |>
    summarize(
      across(
        c(ret_excess_vw, ret_excess_ew, ret_excess_vw_capped),
        list(
          mean = \(x) mean(x) * 12,
          tstat = \(x) mean(x) / (sd(x) / sqrt(n()))
        ),
        .names = "{.col}_{.fn}"
      ),
      .groups = "drop"
    )
}
univariate_sort_table <- bind_rows(
  portfolio_returns |>
    group_by(portfolio) |>
    return_summary() |>
    mutate(portfolio = as.character(portfolio)),
  factor_returns_ag |>
    return_summary() |>
    mutate(portfolio = "L/S")
)
```

The value-weighted long-short portfolio earns 3.4% per year (*t*~statistic of 2.28), consistent with the premium documented by Cooper et al. (2008). The equal-weighted spread is larger, as expected for characteristics that correlate with size. The monotone decline of value-weighted means across quintiles confirms that **tidyfinance** reproduces the literature result from raw WRDS data with a single function call.

Customized specification (bivariate sorts)

The baseline specification above takes nearly every package default and runs a univariate sort to form long-short portfolios. The customized specification below changes both: it overrides several of those defaults through `portfolio_sort_options` and moves from sorting on one variable to sorting on two.

We adopt a specification that combines, in a single call to `implement_portfolio_sort()`, the universe restrictions and breakpoint conventions used in many empirical studies: exclude financials and utilities, exclude firms with non-positive earnings, ignore the smallest 20% of NYSE-listed firms when computing breakpoints, form quintiles on AG within size buckets, use NYSE-only breakpoints, rebalance annually in July, and require at least ten constituents per portfolio.

Bivariate sorts control for a second variable, typically firm size (`mktcap_lag`), when forming portfolios on the main characteristic. We use the dependent variant introduced in Section 2.3.5 and illustrated in Figure 2: stocks are first split into size buckets, then sorted into asset growth portfolios within each bucket, with returns averaged across the size dimension.

```
options_customized <- portfolio_sort_options(
  filter_options = filter_options(
    exclude_financials = TRUE,
    exclude_utilities = TRUE,
    exclude_negative_earnings = TRUE
  ),
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE",
    breakpoints_min_size_threshold = 0.2
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
    breakpoints_exchanges = "NYSE"
  )
)

portfolio_returns_customized <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_customized,
  min_portfolio_size = 10,
  quiet = TRUE
)
factor_returns_customized <- portfolio_returns_customized |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

A single options object (`options_customized`) bundles every methodological choice that distinguishes this specification from the baseline; nothing in the surrounding pipeline changes. Switching from a univariate to a size-controlled dependent bivariate sort is a one-word change in `sorting_method`, and the secondary breakpoint object is supplied alongside the primary one without any additional work. Setting `quiet = TRUE` suppresses the diagnostic messages from `filter_sorting_data()`, which report how many observations each filter drops; setting `quiet = FALSE` (the default) is useful when iterating to track sample-construction effects.

For comparison, we also include a bivariate-dependent sort that uses the default filters and breakpoint conventions, so the only difference from the baseline is the addition of size control. The long-short returns from that specification are appended to the customized one in Table 5.

```
options_bivariate <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
    breakpoints_exchanges = "NYSE"
```

Table 5: Comparison of long-short asset growth returns across three specifications. The baseline is a univariate sort with NYSE breakpoints and no filters. The bivariate-only specification adds a dependent bivariate sort on size to the baseline but otherwise keeps the same universe and breakpoints. The customized specification adds the filter set (financials, utilities, non-positive earnings) and a 20% size threshold on the breakpoints. All rows use annual July rebalancing; means are annualized value-weighted excess returns. Sample: 1974–2024.

Specification	VW mean	VW t statistic
Baseline (univariate, no filters)	0.034	2.280
Bivariate only (size-controlled, no filters)	0.036	3.062
Customized (size-controlled, filtered)	0.026	1.932

```
  )
)
portfolio_returns_bivariate <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_bivariate
)
factor_returns_bivariate <- portfolio_returns_bivariate |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5 decomposes the customized premium into two layers. Adding the size-controlled bivariate sort alone (the bivariate-only row) lifts the annualized premium from 3.4% to 3.6%, with the t~statistic moving from borderline to significant at conventional levels. Controlling for size reveals that most of the AG signal in the cross-section was masked by size dispersion in the univariate sort. Layering the filter set and the 20% breakpoint threshold on top trims the magnitude back to 2.6%, where the spread remains significant at conventional levels: removing financials, utilities, and loss-making firms strips out some of the firms contributing to the unfiltered spread. Across all three specifications, the spread is positive and reflects the negative AG premium reported in Cooper et al. (2008), who finds an annual value-weighted long-short premium of about 12.6% over July 1968 to June 2003.[2]

Researchers who want finer-grained control can call the lower-level portfolio-sorting functions in Table 1 directly. They share signatures and semantics with `implement_portfolio_sort()`, so an analysis prototyped against the wrapper can be unrolled into its components without rewriting the pipeline. This separation matters when custom logic applies at one stage only, for example, computing breakpoints separately within each industry, or from a trailing window of past data rather than the contemporaneous cross-section.

Asset pricing tests

A long-short return is one summary of whether asset growth predicts the cross-section of returns. A complementary view comes from Fama-MacBeth regressions (Fama and MacBeth, 1973), which estimate the average risk premium associated with a characteristic after running cross-sectional regressions period by period. **tidyfinance** implements the standard procedure in `estimate_fama_macbeth()`, which takes a panel and a model formula, fits one cross-sectional ordinary least squares (OLS) regression per period, and aggregates the period-level coefficients into average risk premia with Newey-West standard errors and automatic lag selection.

Here, we winsorize the AG signal at the 1st and 99th percentiles to keep extreme observations from dominating the cross-sectional regressions and pair it with log market capitalization as a size control.

```
fm_data <- sorting_data |>
  drop_na(asset_growth, mktcap_lag, ret_excess) |>
  group_by(date) |>
  mutate(
    asset_growth = winsorize(x = asset_growth, cut = 0.01),
    log_mktcap = log(mktcap_lag)
```

[2] Cooper et al. (2008) sorts into deciles.

Table 6: Fama-MacBeth regression of monthly excess returns on lagged asset growth and lagged log market capitalization. Coefficients are time-series averages of monthly cross-sectional OLS estimates; standard errors are Newey and West (1987) with automatic lag selection. The intercept and slopes on the right-hand-side variables are reported on the monthly scale. Sample: 1974–2024.

Variable	Risk premium	SE	*t* statistic	N (months)
Intercept	0.0168	0.0046	3.6633	606
Asset growth	-0.0055	0.0008	-6.9800	606
Log market cap	-0.0014	0.0004	-3.2775	606

```
  ) |>
  ungroup()

fm_results <- estimate_fama_macbeth(
  data = fm_data,
  model = "ret_excess ~ asset_growth + log_mktcap"
)
```

The negative slope on `asset_growth` in Table 6 lines up with the long-short evidence in Table 4 and Table 5: high-AG firms earn lower subsequent returns, conditional on size. As an external benchmark, Cooper et al. (2008, Table III, Panel A, Model 1) reports a Fama-MacBeth slope on annual asset growth of -0.0922 (t~statistic of -6.52). Converted to a monthly basis, their slope is approximately -0.0077 ($= -0.0922/12$), directly comparable to the magnitude of the AG coefficient in our table.[3] Switching to `vcov = "iid"` or supplying `vcov_options = list(lag = 6, prewhite = FALSE)` would change the standard-error treatment without altering the rest of the call, and adding additional regressors (e.g., a beta estimate from `estimate_betas()`, a profitability measure, an idiosyncratic-volatility control) is a one-line extension to the model formula. The same function therefore covers the standard cross-sectional asset-pricing test that complements the portfolio-sort evidence used throughout the rest of the paper.

5 Summary and discussion

Empirical asset pricing depends on dozens of construction choices that no consensus pins down. **tidyfinance** makes those choices first-class objects in the workflow: each is an argument, each has a documented default, each can be varied without rebuilding the pipeline. Disagreements over construction become empirical questions, not implicit assumptions buried in code.

We will maintain and extend the package on CRAN. At the time of writing, all functions are covered by comprehensive unit tests, with code coverage reported at 100% by Codecov, and each release is accompanied by passing R CMD check results across multiple operating systems, including the latest macOS, Windows, and Ubuntu environments. The companion risk-free rate, hosted on Hugging Face, is refreshed on a scheduled GitHub Actions workflow, so the published series tracks new WRDS releases without manual intervention.

Companion materials at tidy-finance.org integrate **tidyfinance** into open coursework and self-study tracks for graduate finance and asset pricing in R. The site also hosts a reviewed channel through which researchers can publish replications and new construction routines, so the catalog of factors and conventions can grow with the literature rather than freezing at the package's release.

Computational details

We obtained the results in this paper using R 4.6.1 with the following packages: **tidyfinance** 0.8.0, **dplyr** 1.2.1 (Wickham et al., 2026), **tidyr** 1.3.2 (Wickham et al., 2024), **ggplot2** 4.0.3 (Wickham, 2016), and **lubridate** 1.9.5 (Spinu et al., 2024). We install development versions of **tidyfinance** with **pak** NA (Csárdi and Hester, 2024). Figures additionally use **ggforce** 0.5.0 (Pedersen, 2024a), **scales** 1.4.0 (Wickham et al., 2023), and **patchwork** 1.3.2 (Pedersen, 2024b). CRAN hosts R and all packages at `https://CRAN.R-project.org/`. **renv** 1.2.3 (Ushey and Wickham, 2024) ensures full reproducibility of the computational environment. Reproducing the factor return data requires an institutional

[3] Cooper et al. (2008) additionally controls for book-to-market and 6- and 36-month past returns, both negatively correlated with AG.

subscription to WRDS, as the underlying CRSP and Compustat data are proprietary and cannot be redistributed. A pseudo-data path with identical schema is available via `download_data(domain = "Pseudo Data", ...)` for readers without WRDS access; see Section 2.4.1.

Reproducing this manuscript takes about 15 minutes: `rmarkdown::render("tidyfinance.Rmd")` uses the WRDS subset described in Section 2.4.1, or pseudo data of identical schema for readers without WRDS access. Timings reported here are obtained on macOS Tahoe 26.5.1 with a 8-core processor running R 4.6.1; WRDS extracts are not pre-cached, so the 15-minute figure includes the time to download the underlying CRSP and Compustat panels.

We publish all code, data workflows, and documentation at https://github.com/tidy-finance/paper-factor-construction. The code is released under the MIT License, which is compatible with the GNU General Public License (GPL).

Acknowledgments

Stefan Voigt is grateful to the Danish Finance Institute for financial support.

References

P. Akey, A. Z. Robertson, and M. Simutin. Noisy Factors? The Retroactive Impact of Methodological Changes on the Fama–French Factors. *Review of Finance*, 2026. doi: /10.1093/rof/rfag002. Advance access. [p2]

N. Areal. **frenchdata***: Download Data Sets from Kenneth's French Finance Data Library Site*, 2021. R package version 0.2.0. [p2]

W. Armstrong, D. Eddelbuettel, J. Laing, and J. Ooms. **Rblpapi***: R Interface to 'Bloomberg'*, 2024. [p2]

T. G. Bali, R. F. Engle, and S. Murray. *Empirical Asset Pricing: The Cross Section of Stock Returns*. John Wiley & Sons, 2016. [p6, 8]

K. Boudt, J. Cornelissen, S. Payseur, O. Kleen, and E. Sjoerup. **highfrequency***: Tools for Highfrequency Data Analysis*, 2024. [p2]

A. Chen and T. Zimmermann. Open Source Cross-Sectional Asset Pricing. *Critical Finance Review*, 11 (2):207–264, 2022. doi: 10.1561/104.00000112. [p1, 2]

M. J. Cooper, H. Gulen, and M. J. Schill. Asset Growth and the Cross-Section of Stock Returns. *The Journal of Finance*, 63(4):1609–1651, 2008. doi: 10.1111/j.1540-6261.2008.01370.x. [p1, 10, 12, 14, 16, 17]

G. Csárdi and J. Hester. **pak***: Another Approach to Package Installation*, 2024. R package version 0.8.0. [p17]

M. Dancho and D. Vaughan. **tidyquant***: Tidy Quantitative Financial Analysis*, 2026. R package version 1.0.12. [p2]

J. Dick-Nielsen. Liquidity Biases in TRACE. *The Journal of Fixed Income*, 19(2):43–55, 2009. doi: 10.3905/jfi.2009.19.2.043. [p5, 6]

A. Dickerson, P. Mueller, and C. Robotti. Priced Risk in Corporate Bonds. *Journal of Financial Economics*, 150(2):103707, 2023. doi: 10.1016/j.jfineco.2023.103707. [p1, 2]

D. Eddelbuettel, K. Nguyen, and T. Leitch. **RQuantLib***: R Interface to the 'QuantLib' Library*, 2024. [p2]

E. F. Fama and K. R. French. The Cross-Section of Expected Stock Returns. *The Journal of Finance*, 47(2): 427–465, 1992. doi: 10.1111/j.1540-6261.1992.tb04398.x. [p6]

E. F. Fama and K. R. French. Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, 33(1):3–56, 1993. doi: 10.1016/0304-405X(93)90023-5. [p2, 7, 11]

E. F. Fama and K. R. French. Production of U.S. rm-rf, SMB, and HML in the fama-french data library. Chicago Booth Research Paper No. 23-22, Fama-Miller Working Paper, Dec. 2023. [p1, 2]

E. F. Fama and J. D. MacBeth. Risk, Return, and Equilibrium: Empirical Tests. *Journal of Political Economy*, 81(3):607–636, 1973. doi: 10.1086/260061. [p16]

A. Goyal and I. Welch. A Comprehensive Look at the Empirical Performance of Equity Premium Prediction. *The Review of Financial Studies*, 21(4):1455–1508, 2008. doi: 10.1093/rfs/hhm014. [p6]

N. Hautsch, A. J. Menkveld, and S. Voigt. Market responses to a VIX shock. Working Paper, 2026. [p4]

K. Hou, C. Xue, and L. Zhang. Replicating Anomalies. *The Review of Financial Studies*, 33(5):2019–2133, 2020. doi: 10.1093/rfs/hhy131. [p2]

K. Hou, H. Mo, C. Xue, and L. Zhang. An Augmented Q-Factor Model with Expected Growth. *Review of Finance*, 25(1):1–41, 2021. doi: 10.1093/rof/rfaa004. [p4]

T. I. Jensen, B. Kelly, and L. H. Pedersen. Is There a Replication Crisis in Finance? *The Journal of Finance*, 78(5):2465–2518, 2023. doi: 10.1111/jofi.13249. [p1, 2, 4, 8]

W. K. Newey and K. D. West. A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610. [p4]

Ľ. Pástor and R. F. Stambaugh. Liquidity Risk and Expected Stock Returns. *Journal of Political Economy*, 111(3):642–685, 2003. doi: 10.1086/374184. [p4]

T. L. Pedersen. **ggforce***: Accelerating* **ggplot2**, 2024a. R package version 0.4.2. [p17]

T. L. Pedersen. **patchwork***: The Composer of Plots*, 2024b. R package version 1.2.0. [p17]

B. G. Peterson and P. Carl. **PerformanceAnalytics***: Econometric Tools for Performance and Risk Analysis*, 2026. R package version 2.1.0. [p2]

J. A. Ryan and J. M. Ulrich. **quantmod***: Quantitative Financial Modelling Framework*, 2025. R package version 0.4.28. [p2]

C. Scheuch, S. Voigt, and P. Weiss. *Tidy Finance with R*. Chapman and Hall/CRC, 1st edition, 2023. doi: 10.1201/b23237. [p1, 2, 6]

P. Schwarz, D. Walter, and P. Weiss. Rewriting CRSP's History: Impact of Altered Monthly Returns on Asset Pricing. *Journal of Financial and Quantitative Analysis*, page 1–60, 2026. doi: 10.1017/S0022109026102774. [p6]

A. Soebhag, B. Van Vliet, and P. Verwijmeren. Non-standard errors in asset pricing: Mind your sorts. *Journal of Empirical Finance*, 78:101517, 2024. doi: 10.1016/j.jempfin.2024.101517. [p1]

V. Spinu, G. Grolemund, and H. Wickham. **lubridate***: Make Dealing with Dates a Little Easier*, 2024. URL https://lubridate.tidyverse.org. R package version 1.9.4. [p9, 17]

R. F. Stambaugh and Y. Yuan. Mispricing Factors. *The Review of Financial Studies*, 30(4):1270–1315, 2017. doi: 10.1093/rfs/hhw107. [p4]

K. Ushey and H. Wickham. **renv***: Project Environments*, 2024. R package version 1.0.7. [p17]

D. Walter, R. Weber, and P. Weiss. Methodological uncertainty in portfolio sorts. Working Paper, 2024. [p1]

H. Wickham. **ggplot2***: Elegant Graphics for Data Analysis*. Springer-Verlag, New York, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org. [p17]

H. Wickham. The tidy tools manifesto, Nov. 2023. URL https://tidyverse.tidyverse.org/articles/manifesto.html. Accessed: 2026-06-05. [p2]

H. Wickham, T. L. Pedersen, and D. Seidel. **scales***: Scale Functions for Visualization*, 2023. R package version 1.3.0. [p17]

H. Wickham, D. Vaughan, and M. Girlich. **tidyr***: Tidy Messy Data*, 2024. URL https://tidyr.tidyverse.org. R package version 1.3.1. [p9, 17]

H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. **dplyr***: A Grammar of Data Manipulation*, 2026. URL https://dplyr.tidyverse.org. R package version 1.2.1. [p9, 17]

D. Wuertz, T. Setz, Y. Chalabi, and W. Chen. **fPortfolio***: Rmetrics — Portfolio Selection and Optimization*, 2023. [p2]

Christoph Frey
Lancaster University
Management School, Bailrigg
Lancaster LA1 4YX, United Kingdom
ORCiD: 0000-0002-9937-0903
christoph.frey@gmail.com

Christoph Scheuch
Humboldt University of Berlin
School of Business and Economics, Dorotheenstraße 1
10099 Berlin, Germany
ORCiD: 0009-0004-0423-6819
christoph.scheuch@hu-berlin.de

Stefan Voigt
University of CopenhagenDanish Finance Institute
Department of Economics, Øster Farimagsgade 5
1353 Copenhagen K, Denmark
ORCiD: 0000-0001-5619-3161
stefan.voigt@econ.ku.dk

Patrick Weiss
Reykjavík University
Department of Business and Economics, Menntavegur 1
102 Reykjavík, Iceland
ORCiD: 0000-0002-9282-5872
patrickw@ru.is